

TP : Gestion des Sessions HTTP dans Laravel 12

Dans ce TP, nous allons créer une application Laravel simple pour démontrer la gestion des sessions HTTP. Le contexte sera une application web basique qui simule un compteur de visites pour un utilisateur, stocke des préférences temporaires (comme un nom d'utilisateur ou une liste d'équipes), et teste les différentes fonctionnalités des sessions.

Les sessions permettent de stocker des informations persistantes sur l'utilisateur à travers plusieurs requêtes HTTP, en utilisant un backend comme des fichiers ou une base de données.

1 : Création du Projet Laravel

1. Ouvrez un terminal et exécutez la commande pour créer un nouveau projet Laravel 12 :

```
composer create-project laravel/laravel 011-session
```

2. Accédez au répertoire du projet :

```
cd 011-session
```

3. Lancez le serveur de développement :

```
php artisan serve
```

Ouvrez votre navigateur à l'adresse <http://127.0.0.1:8000> pour vérifier que l'application fonctionne (vous devriez voir la page d'accueil par défaut de Laravel).

2 : Configuration des Sessions

Le fichier de configuration des sessions se trouve dans **config/session.php**. Par défaut, Laravel utilise le driver **file** (stockage dans des fichiers), ce qui est adapté pour un développement local.

1. Ouvrez **config/session.php** et vérifiez que le driver est bien sur 'file' (c'est le cas par défaut). Pas de changement nécessaire pour ce TP, mais examinez les options disponibles.

Optionnel : Utiliser le Driver Database pour un Stockage Centralisé Si vous souhaitez tester un backend centralisé (utile pour des applications multi-serveurs), configurez le driver database :

2. Modifiez .env pour configurer une base de données (exemple avec SQLite pour simplicité) :

```
DB_CONNECTION=sqlite
DB_DATABASE=/absolute/path/to/database.sqlite # Créez un fichier database.sqlite
dans database/
```

3. Changez le driver dans config/session.php :

```
'driver' => env('SESSION_DRIVER', 'database'),
```

Modifiez aussi .env : SESSION_DRIVER=database.

4. Créez la table des sessions avec Artisan :

```
php artisan session:table  
php artisan migrate
```

Cela génère une migration et crée la table sessions avec les colonnes suivantes (comme dans le document) :

- id (string, primary)
- user_id (foreignId, nullable, index)
- ip_address (string, 45, nullable)
- user_agent (text, nullable)
- payload (text)
- last_activity (integer, index)

3 : Création d'un Contrôleur et de Routes pour Tester les Sessions

Nous allons créer un contrôleur dédié pour gérer les sessions et définir des routes pour tester chaque fonctionnalité.

1. Créez un contrôleur :

```
php artisan make:controller SessionController
```

2. Ouvrez **app/Http/Controllers/SessionController.php** et importez les classes nécessaires en haut du fichier :

```
namespace App\Http\Controllers;  
use Illuminate\Http\Request;
```

- #### 3. Définissez des méthodes dans le contrôleur pour chaque test (nous les implémenterons au fur et à mesure). Pour l'instant, laissez-les vides.
- #### 4. Ouvrez **routes/web.php** et ajoutez les routes suivantes (supprimez les routes par défaut si nécessaire) :

```
use App\Http\Controllers\SessionController;  
  
Route::get('/session/set', [SessionController::class, 'setData']);  
Route::get('/session/get', [SessionController::class, 'getData']);  
Route::get('/session/all', [SessionController::class, 'getAll']);  
Route::get('/session/has', [SessionController::class, 'checkHas']);  
Route::get('/session/exists', [SessionController::class, 'checkExists']);  
Route::get('/session/missing', [SessionController::class, 'checkMissing']);  
Route::get('/session/push', [SessionController::class, 'pushToArray']);  
Route::get('/session/pull', [SessionController::class, 'pullData']);  
Route::get('/session/increment', [SessionController::class, 'incrementCount']);  
Route::get('/session/decrement', [SessionController::class, 'decrementCount']);  
Route::get('/session/flash', [SessionController::class, 'flashData']);
```

```
Route::get('/session/reflash', [SessionController::class, 'reflashData']);
Route::get('/session/keep', [SessionController::class, 'keepData']);
Route::get('/session/forget', [SessionController::class, 'forgetData']);
Route::get('/session/flush', [SessionController::class, 'flushData']);
Route::get('/session/regenerate', [SessionController::class, 'regenerateId']);
Route::get('/session/invalidate', [SessionController::class,
'invalidateSession']);
```

Ces routes permettront de tester chaque fonction via des requêtes GET simples dans le navigateur.

4 : Tester les Fonctions de Gestion des Sessions

Implémentez et testez chaque méthode pas à pas dans **SessionController.php**. Après chaque implémentation, rechargez le serveur si nécessaire et accédez à l'URL correspondante pour observer le résultat (utilisez **dd()** ou **return** pour afficher les outputs).

4.1 : Stockage des Données (put et session helper)

Implémentez **setData** :

```
public function setData(Request $request)
{
    // Via instance de requête
    $request->session()->put('user.name', 'John Doe');
    $request->session()->put('visit_count', 1);

    // Via helper session
    session(['user.email' => 'john@example.com']);

    return 'Données stockées dans la session.';
}
```

- Test : Accédez à **/session/set**.
- Résultat attendu : Message affiché, et les données sont stockées (vérifiez dans les fichiers de session dans `storage/framework/sessions` ou dans la table DB si driver database).

4.2 : Récupération des Données (get)

Implémentez **getData** :

```
public function getData(Request $request)
{
    // Récupération avec valeur par défaut
    $name = $request->session()->get('user.name', 'Invité');
    $email = session('user.email', 'non@defini.com'); // Via helper
}
```

```
    return "Nom: $name, Email: $email";  
}
```

- Test : Accédez à `/session/get` (après avoir exécuté `/session/set`).
- Résultat attendu : "Nom: John Doe, Email: john@example.com". Si clé absente : valeurs par défaut.

4.3 : Récupérer Toutes les Données (all)

Implémentez `getAll` :

```
public function getAll(Request $request)  
{  
    $data = $request->session()->all();  
    dd($data); // Dump pour afficher le tableau  
}
```

- Test : `/session/all`.
- Résultat attendu : Tableau avec toutes les clés (ex. `['user' => ['name' => 'John Doe', 'email' => ...]]`).

4.4 : Vérifier la Présence (has, exists, missing)

Implémentez `checkHas` :

```
public function checkHas(Request $request)  
{  
    if ($request->session()->has('user.name')) {  
        return 'La clé user.name existe et n\'est pas null.';  
    }  
    return 'La clé user.name n\'existe pas ou est null.';  
}
```

Implémentez `checkExists` (similaire pour `exists`, qui vérifie même si `null`) et `checkMissing` (inverse de `exists`).

- Test : `/session/has`, `/session/exists`, `/session/missing`.
- Résultat attendu : Messages booléens basés sur la présence.

4.5 : Pousser dans un Tableau (push)

Implémentez `pushToArray` :

```
public function pushToArray(Request $request)  
{  
    $request->session()->push('user.teams', 'Developers');  
    $request->session()->push('user.teams', 'Designers');  
    return 'Valeurs poussées dans user.teams.';  
}
```

- Test : **/session/push**, puis vérifiez avec **/session/all**.
- Résultat attendu : **user.teams** devient un tableau ['Developers', 'Designers'].

4.6 : Récupérer et Supprimer (pull)

Implémentez **pullData** :

```
public function pullData(Request $request)
{
    $name = $request->session()->pull('user.name', 'Inconnu');
    return "Nom retiré: $name";
}
```

- Test : **/session/pull** (après set).
- Résultat attendu : Retourne la valeur et la supprime (vérifiez avec **/session/all**).

4.7 : Incrémentation et Décrémentation

Implémentez **incrementCount** :

```
public function incrementCount(Request $request)
{
    $request->session()->increment('visit_count', 1);
    return $request->session()->get('visit_count');
}
```

Implémentez **decrementCount** de manière similaire.

- Test : **/session/increment** plusieurs fois.
- Résultat attendu : Le compteur augmente (ex. 1, 2, 3...).

4.8 : Données Flash

Implémentez **flashData** :

```
public function flashData(Request $request)
{
    $request->session()->flash('status', 'Tâche réussie !');
    return 'Données flash stockées.';
}
```

- Test : **/session/flash**, puis **/session/get** (remplacez get pour vérifier 'status').
- Résultat attendu : Disponible pour la prochaine requête seulement, puis supprimé.

Implémentez **reflashData** pour conserver les flash pour une requête supplémentaire :

```
public function reflashData(Request $request)
{

```

```
$request->session()->reflash();  
return 'Flash reflashés.';  
}
```

Et **keepData** pour conserver des clés spécifiques :

```
public function keepData(Request $request)  
{  
    $request->session()->keep(['status']);  
    return 'Flash spécifiques conservés.';  
}
```

4.9 : Suppression des Données (forget, flush)

Implémentez **forgetData** :

```
public function forgetData(Request $request)  
{  
    $request->session()->forget('user.name');  
    $request->session()->forget(['user.email', 'visit_count']);  
    return 'Clés oubliées.';  
}
```

Implémentez **flushData** pour tout supprimer :

```
public function flushData(Request $request)  
{  
    $request->session()->flush();  
    return 'Session vidée.';  
}
```

- Test : Après set, exécutez **/session/forget** ou **/session/flush**, puis vérifiez avec **/session/all**.

4.10 : Régénération de l'ID de Session

Implémentez **regenerateId** :

```
public function regenerateId(Request $request)  
{  
    $request->session()->regenerate();  
    return 'ID de session régénéré.';  
}
```

Et **invalidateSession** pour régénérer et vider :

```
public function invalidateSession(Request $request)
```

```
{  
    $request->session()->invalidate();  
    return 'Session invalidée.';  
}
```

- Test : **/session/regenerate**. Utile pour la sécurité (ex. contre fixation de session).